Documentación: objeto de dominio Brand

Trabajo de Ingeniería de Software, "Clean Architecture: incorporación de un objeto de dominio". Este documento registra lo que se hizo, el código y la explicación de cada parte.

Resumen

Se agregó al sistema de comercio electrónico (src/ISII.Web) un nuevo objeto de dominio, Brand (marca de un producto). Se puede crear (POST /Brands) y consultar por id (GET /Brands/{BrandId}). Se siguió el mismo estilo que el aggregate Product, que ya existía en el proyecto.

La solución atraviesa tres capas:

Capa	**Qué se hizo**
Dominio	La entidad Brand, su identificador BrandId y la consulta BrandByIdSpec, con las reglas de negocio
Infraestructura	Configuración de la tabla, registro en el contexto de la base y migración
Presentación	DTO, record de respuesta y los dos endpoints, con validación

El objeto Brand

Una marca representa al fabricante o la marca comercial de productos del e-commerce. No duplica ninguna funcionalidad existente (los aggregates previos son Cart, GuestUser, Order y Product).

Campo	**Tipo**	**Regla de negocio**
Id	**BrandId**	Lo asigna la base de datos al guardar; no puede ser negativo
Name	**texto**	Obligatorio, máximo 100 caracteres
Country	**texto**	Obligatorio
FoundedYear	**entero**	Entre 1800 y el año actual

Las reglas están en dos lugares, a propósito: dentro de la entidad (el dominio nunca acepta datos inválidos, vengan de donde vengan) y en el validador del endpoint (devuelve a quien usa la API un mensaje claro antes de ejecutar nada).

3.Archivos creados y modificados

Archivos	Capa	Acción
Domain/BrandAggregate/BrandId.cs	Dominio	Nuevo
Domain/BrandAggregate/Brand.cs	Dominio	Nuevo
Domain/BrandAggregate/Specifications/BrandByIdSpec.cs	Dominio	Nuevo
Infrastructure/Data/Config/BrandConfiguration.cs	Infraestructura	Nuevo
Infrastructure/Data/AppDbContext.cs	Infraestructura	Modificado (se agregó DbSet<Brand> y su using)
Archivo donde se registran los ids para EF Core	Infraestructura	Modificado (se agregó [EfCoreConverter<BrandId>] y su using)
Migrations/..._BrandTable.cs	Infraestructura	Generado con dotnet ef
BrandFeatures/BrandDto.cs y BrandRecord.cs	Presentación	Nuevo
BrandFeatures/Create/CreateBrandEndpoint.cs	Presentación	Nuevo
BrandFeatures/GetById/GetBrandHandler.cs y GetBrandByIdEndpoint.cs	Presentación / aplicación	Nuevos

4. Dominio

Idea central: el dominio es el centro de la arquitectura y no depende de nada de afuera: no sabe de la base de datos ni de HTTP. Cada objeto de dominio vive en su propia carpeta (un *aggregate*).

1. **BrandId.cs**

```
using Vogen;

namespace ISII.Web.Domain.BrandAggregate;

[ValueObject<int>]
99+ references
public readonly partial struct BrandId
{
    1 reference
    public static BrandId New => From(0);

    14 references
    private static Validation Validate(int value)
        => value >= 0 ? Validation.Ok : Validation.Invalid("BrandId must be non-negative.");
}
```

- **Es un value object:** el identificador tiene su propio tipo en lugar de ser un int suelto. Así el sistema no puede confundir el id de una marca con el de un producto.
- **[ValueObject<int>]** es de la librería Vogen, que genera automáticamente métodos como From(...) al compilar. Por eso el editor a veces marca From en rojo hasta que se compila.
- **BrandId.New** vale 0 y significa "marca todavía no guardada". Al guardar, la base de datos asigna el id real.
- **Validate** impide ids negativos.

2. Brand.cs

```
using Ardalis.GuardClauses;

namespace ISII.Web.Domain.BrandAggregate;

22 references
public class Brand : EntityBase<Brand, BrandId>, IAggregateRoot
{
    5 references
    public const int NameMaxLength = 100;
    3 references
    public const int MinFoundedYear = 1800;

    0 references
    private Brand() { }

    3 references
    public Brand(BrandId id, string name, string country, int foundedYear)
    {
        Id = id;
        Name = ValidateName(name);
        Country = Guard.Against.NullOrWhiteSpace(country, nameof(country));
        FoundedYear = Guard.Against.OutOfRange(
            foundedYear, nameof(foundedYear), MinFoundedYear, DateTime.UtcNow.Year);
    }

    1 reference
    public static Brand Create(BrandId id, string name, string country, int foundedYear)
        => new Brand(id, name, country, foundedYear);

    4 references
    public string Name { get; private set; } = string.Empty;
    4 references
    public string Country { get; private set; } = string.Empty;
    4 references
    public int FoundedYear { get; private set; }

    1 reference
    private static string ValidateName(string name)
    {
        Guard.Against.NullOrWhiteSpace(name, nameof(name));
        if (name.Length > NameMaxLength)
            throw new ArgumentException($"Name must not exceed {NameMaxLength} characters.", nameof(name));
        return name;
    }
}
```

- **EntityBase<Brand, BrandId>** es una clase base que ya trae el proyecto; le da a la entidad su Id tipado. IAggregateRoot indica que es la raíz de un aggregate, lo que permite usar repositorios con ella.
- **Las constantes NameMaxLength y MinFoundedYear** definen los límites una sola vez, y se reutilizan en el validador y en la configuración de la base. Así no pueden quedar inconsistentes.

- **El constructor privado vacío** lo usa Entity Framework para reconstruir la marca al leerla de la base.
- **El constructor público aplica las reglas**. Si un dato las rompe, lanza una excepción y la marca no se crea. Guard es de la librería Ardalis.GuardClauses.
- **Create es el método de fábrica para crear marcas nuevas;** solo llama al constructor.
- **private set:** desde afuera las propiedades solo se leen, para que nadie deje una marca en estado inválido.

3. BrandByIdSpec.cs

```
namespace ISII.Web.Domain.BrandAggregate.Specifications;

2 references
public class BrandByIdSpec : Specification<Brand>
{
    1 reference
    public BrandByIdSpec(BrandId brandId) =>
        Query.Where(brand => brand.Id == brandId);
}
```

Una **specification** describe una consulta reutilizable ("la marca con este id") dentro del dominio, sin saber cómo se ejecuta en la base de datos. La infraestructura es la que la ejecuta.

5. Infraestructura

Idea central: la infraestructura depende del dominio (y nunca al revés). Acá se define cómo se guarda el objeto de dominio.

1. BrandConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using ISII.Web.Domain.BrandAggregate;

namespace ISII.Web.Infrastructure.Data.Config;
0 references
public class BrandConfiguration : IEntityTypeConfiguration<Brand>
{
    0 references
    public void Configure(EntityTypeBuilder<Brand> builder)
    {

        builder.Property(entity => entity.Id)
            .HasValueGenerator<VogenIntIdValueGenerator<AppDbContext, Brand, BrandId>>()
            .HasVogenConversion()
            .IsRequired();


        builder.Property(entity => entity.Name)
            .HasMaxLength(Brand.NameMaxLength)
            .IsRequired();


        builder.Property(entity => entity.Country)
            .HasMaxLength(100)
            .IsRequired();


        builder.Property(entity => entity.FoundedYear)
            .IsRequired();


        builder.HasData(
            new Brand(BrandId.From(1), "Acme", "Argentina", 1990),
            new Brand(BrandId.From(2), "Globex", "Brasil", 2005));
    }
}
```

- **Le dice a Entity Framework Core (el ORM)** cómo se corresponde Brand con una tabla: qué columnas tiene, cuáles son obligatorias y qué largo máximo tienen.
- **El generador de valor y HasVogenConversion()** hacen que el BrandId se guarde como un entero y que la base genere el id.
- **HasData** carga dos marcas de ejemplo cada vez que se crea la base, igual que Product.

2. **AppDbContext.cs (cambio)**

```
using ISII.Web.Domain.BrandAggregate;
// ...
public DbSet<Brand> Brands => Set<Brand>();
```

AppDbContext es la representación de la base de datos en el código. Cada DbSet es una tabla. Sin esta línea el proyecto no reconoce la entidad ni la guarda. Las clases de configuración (como BrandConfiguration) no hace falta registrarlas a mano: el contexto las encuentra solo.

3. **Registro del id (EfCoreConverter)**

Se agregó [EfCoreConverter<BrandId>] (más su using) en el archivo donde el proyecto registra los demás ids. Sin este registro, HasVogenConversion() no existe para BrandId y no compila.

4. **Migración:**

dotnet ef migrations add BrandTable --project src/ISII.Web --startup-project src/ISII.Web

Compara las entidades del proyecto con el estado anterior de la base y genera un archivo en **Migrations** con los cambios que faltan: en este caso crear la tabla **Brands** con las columnas **Id, Name, Country y FoundedYear** y los dos datos de ejemplo. Es un archivo generado: no se edita a mano.

Dato: el repositorio se registra en **InfrastructureServiceExtensions** de forma genérica (**IRepository<>** e **IReadRepository<>**), por eso no hizo falta registrar nada específico para Brand.

6. Presentación

Idea central: la capa más externa. Recibe las solicitudes HTTP, las valida, usa el dominio y la infraestructura y arma la respuesta. Se organiza en "features" (BrandFeatures), con una carpeta por caso de uso. Usa la librería FastEndpoints.

1. **DTO y record**

public record BrandDto(BrandId Id, string Name, string Country, int FoundedYear);
public record BrandRecord(int Id, string Name, string Country, int FoundedYear);

- El DTO (Data Transfer Object) lleva los datos entre las capas internas; el Id sigue siendo un BrandId.
- El record es la respuesta que se convierte en JSON hacia afuera, con el Id como entero común.
- Hoy tienen los mismos campos, pero se mantienen separados a propósito: lo que se expone al exterior se decide aparte de lo que tiene el dominio.

2. CreateBrandEndpoint.cs (crear)

```
using FastEndpoints;
using FluentValidation;
using Microsoft.AspNetCore.Http.HttpResults;
using ISII.Web.Domain.BrandAggregate;
using ISII.Web.BrandFeatures;

namespace ISII.Web.BrandFeatures.Create;

5 references
public sealed class CreateBrandRequest
{
    3 references
    public string Name { get; init; } = string.Empty;
    3 references
    public string Country { get; init; } = string.Empty;
    3 references
    public int FoundedYear { get; init; }
}

0 references
public class CreateBrandEndpoint(IRepository<Brand> repository) :
    Endpoint<CreateBrandRequest,
             Results<Created<BrandRecord>, ValidationProblem, ProblemHttpResult>>
{
```

```
    private readonly IRepository<Brand> _repository = repository;
    0 references
    public override void Configure()
    {
        Post("/Brands");
        AllowAnonymous();
        Summary(s =>
        {
            s.Summary = "Create a new brand";
            s.Description = "Creates a new brand with the specified name, country and founded year.";
            s.ExampleRequest = new CreateBrandRequest { Name = "Acme", Country = "Argentina", FoundedYear = 1990 };
            s.ResponseExamples[201] = new BrandRecord(1, "Acme", "Argentina", 1990);

            s.Responses[201] = "Brand created successfully";
            s.Responses[400] = "Invalid request data";
        });

        Tags("Brands");

        Description(builder => builder
            .Accepts<CreateBrandRequest>()
            .Produces<BrandRecord>(201, "application/json")
            .ProducesProblem(400));
    }
    public override async Task<Results<Created<BrandRecord>, ValidationProblem, ProblemHttpResult>>
        0 references
        ExecuteAsync(CreateBrandRequest request, CancellationToken cancellationToken)
    {
        var brand = Brand.Create(BrandId.New, request.Name, request.Country, request.FoundedYear);
        await _repository.AddAsync(brand, cancellationToken);
        await _repository.SaveChangesAsync(cancellationToken);

        var response = new BrandRecord(brand.Id.Value, brand.Name, brand.Country, brand.FoundedYear);
        return TypedResults.Created($"/Brands/{brand.Id.Value}", response);
    }
}
```

```
1 reference
public sealed class CreateBrandValidator : Validator<CreateBrandRequest>
{
    0 references
    public CreateBrandValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty()
            .WithMessage("El nombre de la marca es obligatorio")
            .MaximumLength(Brand.NameMaxLength)
            .WithMessage($"El nombre de la marca no puede superar los {Brand.NameMaxLength} caracteres");

        RuleFor(x => x.Country)
            .NotEmpty()
            .WithMessage("El país es obligatorio");

        RuleFor(x => x.FoundedYear)
            .Must(year => year >= Brand.MinFoundedYear && year <= DateTime.UtcNow.Year)
            .WithMessage($"El año de fundación debe estar entre {Brand.MinFoundedYear} y el año actual");
    }
}
```

El archivo tiene tres partes:

1. **CreateBrandRequest:** la forma del JSON que llega. No lleva Id porque lo asigna la base.
2. **CreateBrandEndpoint:** Configure define la ruta (POST /Brands), que es anónimo y la documentación de la API (resumen, ejemplos y respuestas posibles). ExecuteAsync crea el objeto de dominio con Brand.Create, lo agrega al repositorio, guarda los cambios y devuelve 201 Created con el BrandRecord. El repositorio llega por inyección de dependencias.
3. **CreateBrandValidator:** revisa la request antes de ejecutar el endpoint. Si algo falla, la API responde 400 con el mensaje. Se usa Must(...) para el año para que el año actual se calcule en cada validación y no una sola vez al arrancar.

3. GetBrandHandler.cs (consulta)

```
using ISII.Web.Domain.BrandAggregate;
using ISII.Web.Domain.BrandAggregate.Specifications;

namespace ISII.Web.BrandFeatures.GetById;

16 references
public record GetBrandQuery(BrandId BrandId) : IQuery<Result<BrandDto>>;

3 references
public class GetBrandHandler(IReadRepository<Brand> _repository)
    : IQueryHandler<GetBrandQuery, Result<BrandDto>>
{
    0 references
    public async ValueTask<Result<BrandDto>> Handle(GetBrandQuery request, CancellationToken cancellationToken)
    {
        var spec = new BrandByIdSpec(request.BrandId);
        var entity = await _repository.FirstOrDefaultAsync(spec, cancellationToken);

        if (entity == null) return Result.NotFound();

        return new BrandDto(entity.Id, entity.Name, entity.Country, entity.FoundedYear);
    }
}
```

- **GetBrandQuery** es el mensaje "quiero la marca con este id". Viaja por el mediador (patrón mediador) hasta el handler.
- **GetBrandHandler** busca en la base con un repositorio de solo lectura y la **specification** del dominio. Si no encuentra la marca devuelve **NotFound;** si la encuentra, un BrandDto.

GetBrandByIdEndpoint.cs (consultar por id)

```
using FastEndpoints;
using FluentValidation;
using Microsoft.AspNetCore.Http.HttpResults;
using ISII.Web.Domain.BrandAggregate;
using ISII.Web.Extensions;

namespace ISII.Web.BrandFeatures.GetById;

7 references
public sealed class GetBrandByIdRequest
{
  1 reference
  public const string Route = "/Brands/{BrandId}";
  3 references
  public int BrandId { get; init; }
}

0 references
public class GetBrandByIdEndpoint(IMediator mediator)
  : Endpoint<GetBrandByIdRequest,
             Results<Ok<BrandRecord>,
                     NotFound,
                     ProblemHttpResult>,
             GetBrandByIdMapper>
{
  0 references
  public override void Configure()
  {
    Get(GetBrandByIdRequest.Route);
    AllowAnonymous();

    Summary(s =>
    {
      s.Summary = "Get a brand by ID";
      s.Description = "Retrieves a specific brand by its unique identifier.";
      s.ExampleRequest = new GetBrandByIdRequest { BrandId = 1 };
      s.ResponseExamples[200] = new BrandRecord(1, "Acme", "Argentina", 1990);

      s.Responses[200] = "Brand found and returned successfully";
      s.Responses[404] = "Brand with specified ID not found";
    });
```

```
    Tags("Brands");

    Description(builder => builder
      .Accepts<GetBrandByIdRequest>()
      .Produces<BrandRecord>(200, "application/json")
      .ProducesProblem(404));
  }

  public override async Task<Results<Ok<BrandRecord>, NotFound, ProblemHttpResult>>
    0 references
    ExecuteAsync(GetBrandByIdRequest request, CancellationToken ct)
  {

    var result = await mediator.Send(new GetBrandQuery(BrandId.From(request.BrandId)), ct);

    return result.ToGetByIdResult(Map.FromEntity);
  }
}

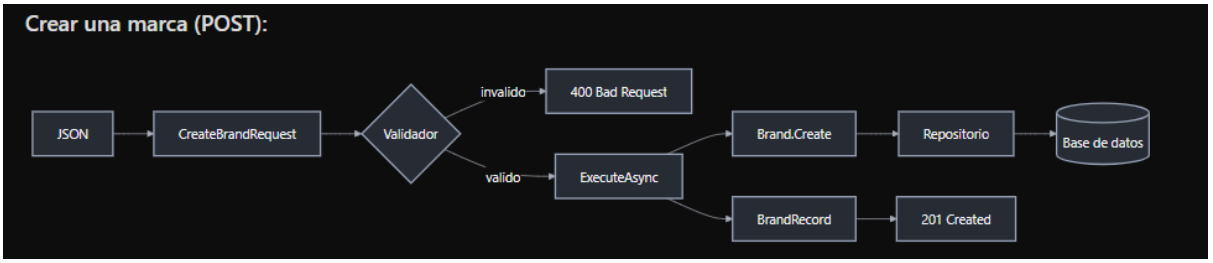

1 reference
public sealed class GetBrandByIdValidator : Validator<GetBrandByIdRequest>
{
  0 references
  public GetBrandByIdValidator()
  {
    RuleFor(x => x.BrandId)
      .GreaterThan(0)
      .WithMessage("El id de la marca debe ser mayor a 0");
  }
}
```

```
1 reference
public sealed class GetBrandByIdMapper
  : Mapper<GetBrandByIdRequest, BrandRecord, BrandDto>
{
  1 reference
  public override BrandRecord FromEntity(BrandDto e)
    => new(e.Id.Value, e.Name, e.Country, e.FoundedYear);
}
```

Cuatro partes:

1. **GetBrandByIdRequest**: el id viene en la URL (/Brands/1).
2. **GetBrandByIdEndpoint:** no consulta la base directamente; le manda la consulta al mediador y convierte el resultado en 200 (con el record) o 404 (no encontrada), con la ayuda de ToGetByIdResult (de la carpeta Extensions).
3. **GetBrandByIdValidator:** el id tiene que ser mayor a 0.
4. **GetBrandByIdMapper:** convierte el BrandDto en el BrandRecord de respuesta.

7. Recorrido de una solicitud



1. Llega un JSON, que se convierte en **CreateBrandRequest.**
2. **CreateBrandValidator** lo revisa. Si falla, se responde 400 y no se ejecuta nada más.
3. ExecuteAsync crea el objeto de dominio con **Brand.Create,** donde actúan las reglas del dominio.
4. El repositorio genérico (inyectado) lo agrega y guarda con Entity Framework Core en la base de datos (SQLite), que le asigna el id.
5. Se arma el **BrandRecord** y se responde 201 Created.

Consultar una marca (GET):

1. **GET /Brands/1** se convierte en **GetBrandByIdRequest;** el validador comprueba que el id sea mayor a 0.
2. **El endpoint** manda un **GetBrandQuery** al mediador, que lo deriva a **GetBrandHandler.**
3. **El handler** busca con **BrandByIdSpec** en el repositorio de lectura.
4. **Si no existe**, se responde 404. Si existe, se arma un **BrandDto,** el mapper lo convierte en **BrandRecord** y se responde 200.

8. Endpoints y pruebas manuales en Bruno

Las siguientes pruebas se realizaron manualmente con Bruno según el registro de trabajo. Los códigos indicados corresponden a los resultados esperados/documentados; si la respuesta observada en Bruno fue distinta, debe registrarse el código rea

Prueba	Método y ruta	Datos y condiciones	Codigo esperado

Crear una marca válida
 	
URL: http://localhost:5737 9/Brands
	POST /Brands	{
		"name": "Marca de ejemplo",
		"country": "Argentina",
		"foundedYear": 2020
		}
			201 Created

Crear una marca inválida
 	
URL: http://localhost:5737 9/Brands
	POST /Brands	{
		"name": "",
		"country": "Argentina",
		"foundedYear": 2020
		}
			400 Bad Request

Consultar una marca existente URL: http://localhost:5737 9/Brands/1	GET /Brands/1	El identificador debe corresponder a una	200 OK
Consultar una marca inexistente URL: http://localhost:5737 9/Brands/999	GET /Brands/999	El identificador 999 no debe existir en la base	404 Not Found

El endpoint de creación responde con 201 cuando la marca se guarda y con 400 si la solicitud no supera la validación. La consulta responde con 200 si encuentra la marca, 404 si no existe y 400 si el identificador no cumple la validación.

La colección de Bruno se encuentra en la carpeta Bruno del repositorio. También existe un archivo api.http con ejemplos de solicitudes HTTP para probar los endpoints desde Visual Studio Code.

9. Pruebas automatizadas

El proyecto incluye pruebas para comprobar las reglas del dominio, las validaciones de entrada y la lógica de consulta.

Archivo de prueba	Qué verifica
BrandTest.cs	La entidad Brand y sus reglas de negocio.
CreateBrandValidatorTest.cs	La validación de datos válidos e inválidos al crear una marca.
GetBrandByIdValidatorTest.cs	La validación del identificador usado para consultar una marca.
GetBrandHandlerTest.cs	La lógica del handler de consulta de una marca.

Según el registro de trabajo, se ejecutó la suite completa y se obtuvieron 25 pruebas aprobadas y 0 errores. Este total incluye pruebas preexistentes de Product; no corresponde únicamente a Brand.

Para volver a ejecutar las comprobaciones, abrir una terminal en la solución y ejecutar:
dotnet build
dotnet test

La compilación comprueba que el código construya correctamente; dotnet test ejecuta las pruebas automatizadas. Si se modifica el proyecto, conviene volver a ejecutar ambos comandos y actualizar el resultado documentado.

10. Migración de base de datos

La migración BrandTable se generó con el siguiente comando:

dotnet ef migrations add BrandTable --project src/ISII.Web --startup-project src/ISII.Web

La migración describe la creación de la tabla Brands y sus columnas. Es un archivo generado por Entity Framework Core y no se debe editar manualmente salvo que exista una razón técnica concreta.